Problema 2: Detección de fraude usando técnicas de aprendizaje supervisado

INFO1185: Inteligencia Artificial

Prof. Dr. Ricardo Soto Catalán

09 de Octubre de 2026

1. Descripción general

La detección automática de transacciones fraudulentas es un problema relevante en el ámbito financiero. Las organizaciones deben identificar actividades sospechosas entre un gran número de transacciones legítimas, en un contexto donde los fraudes son poco frecuentes.

En este proyecto, los equipos deberán implementar y comparar modelos de clasificación supervisada para detectar transacciones fraudulentas. El criterio prioritario será **minimizar la tasa de falsos negativos**, es decir, evitar que una transacción fraudulenta sea clasificada como legítima.

El trabajo contempla preprocesamiento, selección de características, ajuste de hiperparámetros mediante validación cruzada y análisis de estrategias para compensar el desbalance. Además, se implementará una versión propia de Random Forest para comprender el funcionamiento del bagging y la combinación de árboles de decisión.

El proyecto se realizará **en parejas**. La evaluación considerará el código desarrollado y la presentación de los resultados mediante PowerPoint o una herramienta similar, junto con una defensa oral en la que deberán participar ambos integrantes de la dupla. **No se solicitará informe escrito.**

2. Objetivos

1. Explorar un conjunto de datos reales y comprender las implicancias del desbalance de clases.
2. Implementar estrategias de preprocesamiento y selección de características, evitando fuga de información.
3. Entrenar y comparar los siguientes clasificadores:
   - k-Nearest Neighbors (k-NN).
   - Árbol de Decisión.
   - Support Vector Machine (SVM).

- Random Forest de implementación propia.

4. Comprender el bootstrap y el bagging mediante la implementación de un ensamble de árboles de decisión.
5. Seleccionar hiperparámetros mediante validación cruzada estratificada, priorizando la detección de fraudes.
6. Analizar los efectos de una estrategia de compensación del desbalance sobre el desempeño de los clasificadores.
7. Comunicar e interpretar los resultados mediante una presentación y una defensa oral.

3. Conjunto de datos

Se utilizará el conjunto *Credit Card Fraud Detection*, disponible en Kaggle.

- **Número de registros:** 284 807 (Utilizar al menos 10 000).
- **Número de fraudes:** 492, aproximadamente el 0,172 % del total.
- **Variables predictoras:**
  - `Time`: tiempo transcurrido desde la primera transacción del conjunto.
  - `Amount`: importe de la transacción.
  - `V1`–`V28`: variables obtenidas mediante una transformación PCA.
- **Variable objetivo:** `Class`, donde 0 representa una transacción legítima y 1 una transacción fraudulenta.

Las variables transformadas no poseen un significado semántico directo disponible. En este proyecto no se aplicará reducción de dimensionalidad mediante PCA; se trabajará con **selección de características**.

4. Instrucciones técnicas

4.1. Exploración y preprocesamiento

1. Identificar el número de registros, características, valores faltantes y posibles duplicados. Justificar las decisiones de limpieza adoptadas.
2. Determinar la cantidad y proporción de ejemplos de cada clase.
3. Separar las variables predictoras de la variable objetivo. La variable `Class` no debe utilizarse como entrada de los clasificadores.
4. Dividir los datos en entrenamiento (70 %) y prueba (30 %), utilizando una partición estratificada y una semilla fija.
5. Aplicar escalado o normalización cuando corresponda, justificando su uso según el clasificador.

Todos los experimentos deberán utilizar la misma partición de entrenamiento y prueba. El conjunto de prueba conservará la distribución original de clases y permanecerá reservado hasta la evaluación final.

4.2. Selección de características

- Aplicar al menos una técnica de selección basada en modelos, criterios estadísticos o métodos de búsqueda.
- Justificar la técnica utilizada y registrar las características seleccionadas al ajustar el procedimiento final con los datos de entrenamiento.
- Integrar la selección dentro de la validación cruzada. No se permite seleccionar características previamente utilizando todos los datos.

Se pueden utilizar técnicas como información mutua, selección basada en importancia de variables o Sequential Forward Selection.

4.3. Implementación propia de Random Forest

Se deberá implementar un Random Forest utilizando árboles de decisión como modelos base. Se permite utilizar `DecisionTreeClassifier` de Scikit-learn.

No se permite utilizar `RandomForestClassifier` ni `BaggingClassifier` para construir el ensamble solicitado. No es necesario implementar desde cero el algoritmo de construcción de cada árbol.

La implementación deberá:

1. Generar una muestra bootstrap para cada árbol mediante muestreo aleatorio con reemplazo. Cada muestra tendrá el mismo tamaño que el conjunto disponible para entrenar el ensamble.
2. Entrenar un árbol independiente con cada muestra bootstrap.
3. Considerar un subconjunto aleatorio de características en cada división mediante el parámetro `max_features` del árbol base.
4. Combinar las predicciones mediante votación mayoritaria. Utilizar un número impar de árboles o definir una regla explícita para resolver empates.
5. Permitir configurar, al menos, el número de árboles, la profundidad máxima, la cantidad de características consideradas por división y la semilla aleatoria.
6. Utilizar semillas distintas por árbol, derivadas de una semilla general, para asegurar diversidad y reproducibilidad.
7. Organizar el entrenamiento y la predicción mediante funciones o métodos claramente identificados.

La selección aleatoria de características por división es necesaria para distinguir Random Forest del bagging de árboles que consideran todas las características.

La dupla deberá explicar cómo se generan las muestras bootstrap, por qué pueden contener registros repetidos y cómo se obtiene la predicción final del ensamble.

4.4. Tratamiento del desbalance

Para cada uno de los cuatro clasificadores se deberán comparar dos condiciones:

1. Entrenamiento sin una estrategia explícita de compensación del desbalance.
2. Entrenamiento con al menos una estrategia de compensación, como submuestreo, sobremuestreo aleatorio, generación de datos sintéticos o ponderación de clases, cuando el modelo lo permita.

Se puede utilizar una misma estrategia de remuestreo para todos los clasificadores. Si se utilizan estrategias distintas, se deberá justificar la decisión.

Documentar los parámetros de la estrategia y, cuando corresponda, la cantidad de ejemplos por clase antes y después de aplicarla.

El remuestreo se aplicará exclusivamente a los datos de entrenamiento de cada partición de validación cruzada. Los conjuntos de validación y prueba no deberán balancearse.

En Random Forest, las muestras bootstrap se generarán a partir del conjunto de entrenamiento disponible en cada condición experimental.

4.5. Entrenamiento y ajuste de hiperparámetros

1. Entrenar los cuatro clasificadores con y sin compensación del desbalance, obteniendo al menos ocho configuraciones experimentales finales.
2. Utilizar validación cruzada estratificada de cinco particiones dentro del conjunto de entrenamiento.
3. Mantener las mismas particiones de validación para comparar los modelos y las condiciones.
4. Realizar búsqueda de hiperparámetros mediante Grid Search, Random Search o un procedimiento propio equivalente.
5. Definir un espacio de búsqueda acotado y justificar los hiperparámetros considerados.
6. Seleccionar las configuraciones que maximicen la sensibilidad media de la clase fraude, equivalente a minimizar la tasa media de falsos negativos. En caso de empate, utilizar la precisión de la clase fraude como criterio secundario.
7. Registrar los hiperparámetros seleccionados, la sensibilidad media y su desviación estándar en validación cruzada.

En Random Forest, el procedimiento de búsqueda deberá evaluar el ensamble implementado por la dupla.

El escalado, la selección de características y el tratamiento del desbalance deberán ajustarse utilizando únicamente los datos de entrenamiento de cada partición. Se recomienda utilizar pipelines o una organización equivalente.

El conjunto de prueba no se utilizará para seleccionar características, hiperparámetros, estrategias de compensación ni umbrales de decisión.

4.6. Evaluación final

Una vez seleccionadas las configuraciones mediante validación cruzada, entrenarlas nuevamente con el conjunto completo de entrenamiento y evaluarlas en el conjunto de prueba.

Para cada modelo y condición se deberá reportar valor medio y desviación estándar de:

- Sensibilidad.
- Especificidad.
- Precisión.
- F1-score.
- Exactitud.
- Tasa de falsos negativos.

La clase positiva será siempre el fraude. La tasa de falsos negativos se define como:

$$\text{FNR} = \frac{FN}{TP + FN} = 1 - \text{Sensibilidad}.$$

Los resultados de validación y prueba deberán presentarse por separado. No se realizarán nuevos ajustes a partir de los resultados obtenidos en prueba.

4.7. Análisis de resultados

La presentación deberá responder, al menos, las siguientes preguntas:

1. ¿Por qué la exactitud puede resultar engañosa en este problema? ¿Qué exactitud y sensibilidad tendría un clasificador que siempre predice transacciones legítimas?
2. ¿Cómo cambia la detección de fraudes al compensar el desbalance? ¿Qué ocurre con las falsas alarmas?
3. ¿Qué diferencias se observan entre el árbol individual y el Random Forest propio? ¿Qué aporta la combinación de árboles?
4. ¿Qué configuración fue seleccionada mediante validación cruzada y qué desempeño alcanzó en prueba?
5. ¿Qué limitaciones presenta la solución? ¿Qué consecuencias tendría clasificar todas las transacciones como fraudulentas?

El análisis de falsos positivos no modifica el criterio prioritario de minimizar los falsos negativos. Su propósito es comprender las consecuencias operativas de los resultados.

5. Entregables

1. **Código:** notebook o archivos Python que permitan reproducir el procedimiento completo, incluyendo la implementación propia de Random Forest.
2. **Instrucciones de ejecución:** archivo README breve con dependencias, versiones principales, ubicación esperada del dataset, orden de ejecución y semillas utilizadas. No es necesario adjuntar el conjunto de datos.
3. **Presentación:** archivo PowerPoint o equivalente con la metodología, resultados, análisis y conclusiones. Si se utiliza una herramienta en línea, entregar también una copia exportada a PDF.
4. **Defensa oral:** exposición de 10–12 minutos por dupla, seguida de preguntas.

Los resultados y su análisis deberán formar parte de la presentación. Los comentarios del código se utilizarán para explicar la implementación y no reemplazarán la presentación.

No se solicitará informe escrito.

La entrega se realizará en Blackboard.

Fecha de entrega: 26 de octubre a las 23.59 hrs.

Fecha de defensa: Reserva de citas entre el 27 y 28 de octubre.

6. Estructura de la presentación

Se sugiere utilizar entre 10 y 12 diapositivas principales, organizadas de la siguiente manera:

1. **Portada y objetivo (1 diapositiva):** título, integrantes y objetivo del proyecto, destacando la prioridad de minimizar falsos negativos.
2. **Exploración de los datos (1 diapositiva):** características generales, distribución de clases y desafíos del desbalance.
3. **Metodología (1–2 diapositivas):** esquema del procedimiento, división de datos, preprocesamiento, selección de características, compensación del desbalance y validación cruzada. Explicar cómo se evitó la fuga de información.
4. **Random Forest propio (1–2 diapositivas):** esquema o pseudocódigo del bootstrap, entrenamiento de árboles, selección aleatoria de características y votación. Explicar la función del bagging y los parámetros configurables.
5. **Selección de hiperparámetros (1 diapositiva):** resumen del espacio de búsqueda, configuraciones seleccionadas y sensibilidad media con su desviación estándar en validación cruzada.
6. **Resultados en prueba (2 diapositivas):** tablas comparativas de los cuatro clasificadores, con y sin compensación, incluyendo las métricas solicitadas. Complementar con gráficos o matrices de confusión que destaquen los hallazgos principales.

7. **Análisis crítico (1 diapositiva):** efecto de compensar el desbalance, relación entre fraudes detectados y falsas alarmas, comparación entre árbol y ensamble, y limitaciones de la exactitud.

8. **Conclusiones y limitaciones (1 diapositiva):** configuración seleccionada, desempeño final, principales aprendizajes, limitaciones y mejoras.

Las tablas extensas, las matrices de confusión restantes y otros resultados complementarios podrán incluirse como diapositivas de respaldo. Todas las métricas solicitadas deberán estar disponibles en la presentación principal o en sus diapositivas de respaldo.

Ambos integrantes deberán participar en la exposición y conocer la totalidad del desarrollo. Durante la defensa se podrán formular preguntas a cualquiera de ellos y solicitar la revisión o ejecución de partes del código.

7. Evaluación

La evaluación considerará el código y la presentación, con un máximo conjunto de 100 puntos, y una defensa oral de la dupla que determinará un factor de ponderación.

7.1. Rúbrica del código: 40 puntos

Criterio	**Puntaje máximo**
Preprocesamiento y selección de características correctamente implementados, sin fuga de información.	8
Implementación propia de Random Forest: bootstrap, aleatoriedad de características, árboles independientes, votación y parámetros configurables.	14
Implementación de los cuatro clasificadores, comparación con y sin compensación, validación cruzada y búsqueda de hiperparámetros.	12
Organización, documentación técnica, instrucciones de ejecución y reproducibilidad.	6
Total	**40**

7.2. Rúbrica de la presentación: 60 puntos

Criterio	Puntaje máximo
Contextualización del problema y exploración de datos: objetivo, distribución de clases y desafíos.	5
Explicación y justificación de la metodología: preprocesamiento, selección, compensación y validación.	10
Explicación del Random Forest propio y del papel del bootstrap, bagging y votación.	10
Presentación completa y clara de hiperparámetros, resultados de validación y resultados de prueba.	15
Análisis crítico: falsos negativos, falsas alarmas, efecto del desbalance y comparación árbol–ensamble.	10
Conclusiones fundamentadas y limitaciones.	5
Claridad visual, organización, uso del tiempo y participación de ambos integrantes.	5
Total	**60**

Para cada criterio de ambas rúbricas se aplicará uno de los siguientes factores a su puntaje máximo:

- **Excelente (1,00):** cumplimiento completo, correcto y fundamentado.
- **Satisfactorio (0,66):** cumple los aspectos principales, con omisiones menores.
- **Insuficiente (0,33):** desarrollo parcial, errores relevantes o justificación limitada.
- **Deficiente (0,00):** componente ausente o incorrecto.

7.3. Defensa oral de la dupla

El profesor realizará cinco preguntas, con un máximo de 10 puntos por pregunta y 50 puntos en total. Las preguntas se distribuirán entre ambos integrantes y podrán incluir repreguntas para verificar la comprensión del trabajo.

Se evaluará la capacidad de explicar los conceptos, relacionarlos con el código y fundamentar los resultados.

Nivel	Descripción	Puntos
Excelente	Respuesta correcta, fundamentada y vinculada con la implementación y los resultados.	9–10
Muy bueno	Explicación correcta de los aspectos principales, con omisiones menores.	7–8
Satisfactorio	Comprensión básica, con explicaciones parciales o conexiones limitadas con el proyecto.	5–6
Insuficiente	Respuesta vaga o con errores conceptuales relevantes.	3–4
Deficiente	No responde o demuestra desconocimiento del procedimiento y sus fundamentos.	0–2

7.4. Cálculo del puntaje final

Sea P_C el puntaje del código, sobre 40 puntos; P_P el puntaje de la presentación, sobre 60 puntos; y P_D el puntaje de la defensa, sobre 50 puntos.

El factor de ponderación de la defensa será:

$$FP = \frac{P_D}{50}.$$

El puntaje final de la dupla se calculará como:

$$P_{\text{final}} = (P_C + P_P) \cdot FP.$$

El puntaje final, entre 0 y 100, se convertirá a nota mediante la escala de calificación de la asignatura.